Document number: P1069R1
Date: 2018-11-25
Reply-To:
Mike Spertus, Symantec (mike_spertus@symantec.com)
Walter E. Brown (webrown.cpp@gmail.com)
Stephan T. Lavavej (stl@exchange.microsoft.com)
Audience: {Library Evolution, Library} Working Group

Changes from previous version

Now focuses entirely on inferencing heap objects. The remaining portions of P1069R0 may be considered later in separate papers.

Inferencing heap objects

While class template argument deduction can be used for objects with static and automatic duration, it generally cannot be used for creating objects with dynamic duration as the following attempts show:

```
optional o(5); // OK. Static duration. optional<int>

// Try to create dynamic duration object.
auto o1 = new optional(5);  // Already violates both Core Guidelines R.3 and R.11

auto o2 = unique_ptr(o1); // Oops, ill-formed. Can't deduce unique_ptr from raw ptr
```

We propose simply making dynamic object creation with `make_unique`, `make_shared`, and `allocate_shared` work just like the normal way one would translate a static and automatic declaration to a dynamic declaration

```
int i1(5);                       // Static
auto i2 = make_unique<int>(5);   // Make dynamic by moving decl-specifier to make_unique template argument
option o1(5);
auto o2 = make_unique<optional>(5); // Again, make dynamic by moving decl-specifier to make_unique template argument
```

Implementing this can be as simple as the following code, which can be seen working on Wandbox

```
template<template <typename ...U> typename T, typename ...A>
auto make_unique(A&& ...a) {
    return std::unique_ptr<decltype(T(std::forward<A>(a)...))>(new T(std::forward<A>(a)...));
}
```

Notes:

- We only provide overloads for creating non-array objects as the other examples are not natural to deduce
- For simplicity, we do not propose supporting the case of non-type template parameters as it is far less frequent and seems far more complex. If and only if library implementers feel that there are no implementation concerns for non-type template parameters, then that should be supported as well
- It is worth pointing out that because this code exactly matches *mutatis mutandi* the automatic duration declaration case, it should have no cause to be in any way more dangerous or confusing that what is supported for objects created with static or automatic duration

Wording

```
//23.11.1, class template unique_ptr
template<class T> struct default_delete;
template<class T> struct default_delete<T[]>;
template<class T, class D = default_delete<T>> class unique_ptr;
template<class T, class D> class unique_ptr<T[], D>;
template<class T, class... Args> unique_ptr<T>
  make_unique(Args&&... args);  // T is not array
template<template <class ...U> class T, class ...Args>
  auto make_unique(Args&&... args) -> unique_ptr<decltype(T(std::forward<Args>(args)...))>;
template<class T> unique_ptr<T>
  make_unique(size_t n);  // T is U[]

//23.11.3.6, shared_ptr creation
template<class T, class... Args>
  shared_ptr<T> make_shared(Args&&... args);  // T is not array
template<template <class ...U> class T, class ...Args>
  auto make_shared(Args&&... args) -> shared_ptr<decltype(T(std::forward<Args>(args)...))>;
template<class T, class A, class... Args>
  shared_ptr<T> allocate_shared(const A& a, Args&&... args); // T is not array
template<template <class ...U> class T, class A, class ...Args>
  auto allocate_shared(const A&, Args&&... args) -> shared_ptr<decltype(T(std::forward<Args>(args)...))>;

23.11.1.4 unique_ptr creation  [unique.ptr.create]
template<class T, class... Args> unique_ptr<T> make_unique(Args&&... args);
```

```
  Remarks: This function shall not participate in overload resolution unless T is not an array.2
  Returns: unique_ptr<T>(new T(std::forward&Args>(args)...)).
template<template <class ...U> class T, class ...Args>
auto make_unique(Args&&... args) -> unique_ptr<decltype(T(std::forward<Args>(args)...))>;
  Returns: unique_ptr<decltype(T(std::forward<A>(a)...))>(new T(std::forward<A>(a)...))
```

23.11.3.6 shared_ptr creation [util.smartptr.shared.create]

The common requirements that apply to all make_shared and allocate_shared overloads, unless specified otherwise, are described below.

```
template<class T, ...>
  shared_ptr<T> make_shared(args);
template<template <class ...U> class T, class ...Args>
  auto make_shared(Args&&... args) -> shared_ptr<decltype(T(std::forward<Args>(args)...))>;
template<class T, class A, ...>
  shared_ptr<T> allocate_shared(const A& a, args);
template<template <class ...U> class T, class A, class ...Args>
  auto allocate_shared(const A&, Args&&... args) -> shared_ptr<decltype(T(std::forward<Args>(args)...))>;
```